

This document serves as a reference point for all files and deliverables related to the 2025 summer BEL Fellowship with Jared Umphress and Isa Elias . Please contact them with any questions.

Jared – jaredumphress@ucsb.edu

Isa – gelias@ucsb.edu

Link to Dashboard

<https://isa-elias.shinyapps.io/wblt-pasture-monitoring/>

Decision Document

The Decision Document is a written record of all the main decisions made during the project, as well as the options we chose from.

[IrriWatch Decision Doc](#)

Next Steps

Full descriptions can be read in the IrriWatch Decision Doc.

- ☐ Implement daily processing script in AWS Lambda
- ☐ Add units to legend – see units.txt file in pasture-monitoring-dashboard-data bucket
- ☐ Compare one pasture between different water years on one trendline plot and histogram
- ☐ Add opacity slider for raster display
- ☐ Create a GIF of landscape changes over time
- ☐ Add pasture grazing dates (Survey123)
- ☐ Combine satellite data and weather data
- ☐ Purchase more historic data from IrriWatch to see long term trends
- ☐ Display histogram distributions as percent occurrence in addition to just frequency
- ☐ Fix histogram date refresh – currently not updating to new file when date is changed

GitHub Repository for Dashboard/Data Processing Scripts

[GitHub repo link](#) - Note Kyle is collaborator, can transfer ownership to him as desired.

In order to run shiny app locally, you will need a .Renvron file with AWS credentials. In the project root folder, create a file called “.Renvron” – mind the dot – with just the following information:

AWS_ACCESS_KEY_ID= *enter access key id here*

AWS_SECRET_ACCESS_KEY= *enter secret access key here*

AWS_DEFAULT_REGION= us-west-1

BUCKET_NAME= pasture-monitoring-dashboard-data

Note that the access key id and secret can be found [here](#). If deprecated, they will no longer work, but fret not, they can be easily replaced with the new keys that were generated.

Automated Python Scripts

There are four python scripts, and associated Jupyter notebooks, contained within the folder [Python Scripts](#).

1. `daily_irriwatch_processing.py` – **this is the master script that should be automated for daily runs**. It:
 - a. Fetches yesterday and day before yesterday's data from IrriWatch and puts that in S3 bucket `irriwatch-raw-files`
 - b. Merges the rasters together to form one raster, which is stored in the bucket `pasture-monitoring-dashboard` as *`rast_date`*
 - c. Downloads the 2025 shapefile pasture boundaries, divides the merged raster along those boundaries, calculates the mean and variance for each parameter for each pasture, and adds that information to the shapefile. It also stores the list of pixels for each pasture/parameter combination for use in the histogram. This new shapefile with pasture information is stored as *`vecPixels_date`* in the `pasture-monitoring-dashboard` bucket
 - d. Adds the mean of each parameter for each pasture to the timeseries csv file in the timeseries folder for each WY, stored as *`timeseries_wy`* in the `pasture-monitoring-dashboard` bucket

If `overwrite` is set to `True`, any existing dates will be overwritten if called. Can also run a manual date range.

2. `fetch_iw_data_custom_dates.py` – fetches IW data for custom date range with optional `overwrite`.
3. `fetch_iw_layer_files.py` – fetches just the `layer.txt` file from IW.
4. `process_irriwatch_data_custom_dates.py` – processes custom date range with optional `overwrite`.
5. `process_timeseries_file.py` – processes the timeseries file for a given WY with optional `overwrite`.

Note the ReadMe file. Run the lines in your terminal to set the access keys necessary to read/write to/from the S3 buckets used in the scripts.

AWS Storage

- [Link to S3 storage](#)
- [AWS Guide](#)
- [Link to access key and secret key](#)

UI Google Doc Writing

[User Interface Doc](#)